

Marmopedras

Sistema de Controle de Estoque

Documentação Técnica

Projeto Marmopedras

7 de outubro de 2026

Resumo

Este documento descreve o sistema **Marmopedras**, uma aplicação web para controle de estoque de materiais de uma marmoraria. O front-end é desenvolvido em Next.js/React e o back-end é executado no Back4App como Cloud Code (Parse Server), concentrado em um único arquivo JavaScript. São apresentados a arquitetura, o modelo de dados, as funções do servidor, as regras de negócio, a integração entre as camadas, o procedimento de implantação e as limitações conhecidas.

Sumário

1	Introdução	3
2	Visão geral da arquitetura	3
2.1	Tecnologias	3
3	Funcionalidades	4
4	Estrutura do projeto	4
5	Modelo de dados	5
5.1	_User	5
5.2	Classes de estoque	5
5.3	Movimentacao	5
5.4	Lead	5
6	Cloud Functions	6
6.1	Tratamento de erros	6

7	Regras de negócio	7
7.1	Status do material	7
7.2	Entrada	7
7.3	Saída	7
7.4	Transferência	7
7.5	Histórico	7
8	Autenticação e segurança	7
9	Integração do front-end	8
10	Instalação e implantação	8
10.1	Back-end (Back4App)	8
10.2	Front-end	9
10.3	Problemas comuns	9
11	Limitações e trabalhos futuros	9
12	Conclusão	10

1 Introdução

A Marmopedras comercializa revestimentos, argamassas, rejuntas, impermeabilizantes, adesivos e pedras naturais. O controle desses materiais, distribuídos em vários depósitos e retirados para diferentes obras, exige registro confiável de cada movimentação e visão rápida do saldo disponível.

O sistema tem como objetivos:

- registrar entradas, saídas e transferências de materiais;
- mostrar o saldo por material e por depósito, com alerta de estoque baixo ou crítico;
- manter o histórico auditável de movimentações, com responsável, data e hora;
- oferecer um pré-atendimento automatizado ao cliente (chatbot de WhatsApp) que gera *leads* para a equipe de vendas;
- restringir o acesso a usuários autenticados.

2 Visão geral da arquitetura

O sistema segue uma arquitetura em duas camadas. O navegador executa a aplicação Next.js e se comunica com o Back4App por meio de chamadas HTTPS às *Cloud Functions*, que concentram todas as regras de negócio e o acesso ao banco de dados.

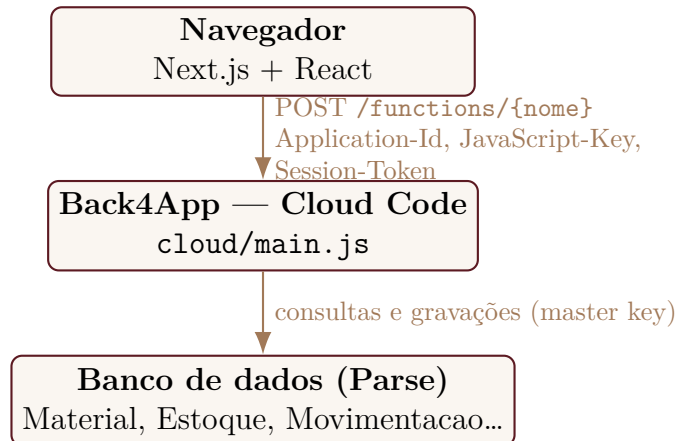


Figura 1: Arquitetura do sistema.

2.1 Tecnologias

Camada	Tecnologia
Front-end	Next.js 15, React 19, JavaScript/JSX
Estilização	Tailwind CSS v4 e CSS global
Back-end	Back4App Cloud Code (Parse Server), JavaScript puro
Banco de dados	Banco gerenciado do Back4App (Parse)
Autenticação	Parse.User com token de sessão

3 Funcionalidades

A Tabela 1 relaciona cada tela do sistema à sua função e à Cloud Function utilizada.

Tela	Descrição	Função(ões)
Login / Cadastro	Criação de conta e autenticação	register, login, me
Painel Geral	Indicadores do dia, últimas movimentações e materiais críticos	painel
Pesquisar Materiais	Busca por nome/código, categoria e status	listarMateriais, categorias
Registrar Entrada	Adiciona material ao estoque de um depósito	registrarEntrada, ultimasMovimentacoes
Registrar Saída	Retira material para uma obra	registrarSaida
Transferência	Move material entre depósitos	registrarTransferencia
Distribuição	Estoque por depósito	distribuicao
Histórico	Movimentações com filtros e paginação	listarMovimentacoes
Chatbot WhatsApp	Fluxo de pré-atendimento que grava o <i>lead</i>	salvarLead

Tabela 1: Telas e funções do servidor.

4 Estrutura do projeto

```

app/
  api.js           cliente das Cloud Functions
  App.js           layout, navegação e Painel Geral
  components/Icons.jsx
  pages/
    Auth.jsx
    ChatbotWhatsApp.jsx
    DistribuicaoEstoque.jsx
    HistoricoMovimentacoes.jsx
    PesquisarMateriais.jsx
    RegistrarEntrada.jsx
    RegistrarSaida.jsx
    Transferencia.jsx
  globals.css, layout.js, page.js
backend/
  cloud/main.js    todas as funções do servidor
docs/
  documentacao.tex este documento
public/
  imagens
.env.example       modelo das variáveis de ambiente

```

5 Modelo de dados

As classes são criadas automaticamente pelo Parse no primeiro registro salvo. As tabelas seguintes descrevem os campos de cada uma.

5.1 _User

Campo	Descrição
username, email	e-mail do usuário (em minúsculas)
password	senha (armazenada com <i>hash</i> pelo Parse)
nome, telefone, empresa	dados de contato
cargo	padrão “Funcionário”

5.2 Classes de estoque

Classe	Campos
Deposito	id_ (A–D), nome, capacidade
Material	cod (MAT-001...), nome, categoria, unidade, minimo
Estoque	cod, deposito, qtd — saldo de um material em um depósito

5.3 Movimentacao

Registra cada operação de estoque.

Campo	Descrição
tipo	Entrada, Saída ou Transferência
cod, material, unidade	identificação do material
quantidade, qtd	valor numérico e texto formatado (+80 m ² , -20 sacos)
local	depósito, “origem → obra” ou “origem → destino”
obra	destino (somente saídas)
de, para	depósitos (somente transferências)
nota, obs	nota fiscal e observações
responsavel	usuário (ou nome informado na saída)
data, hora	AAAA-MM-DD e HH:MM, fuso America/Recife

5.4 Lead

Pré-atendimento do chatbot: nome, localizacao, tipoObra, projetoMedidas, material, corEstilo, telefone e status (inicia como novo).

6 Cloud Functions

Todas as funções estão em `backend/cloud/main.js`. As marcadas como *logado* exigem token de sessão válido; sem ele retornam o código Parse 209.

Função	Acesso	Parâmetros	Retorno
<code>register</code>	público	nome, email, telefone, empresa, cargo, senha	token e usuário
<code>login</code>	público	email, senha	token e usuário
<code>me</code>	logado	—	usuário atual
<code>painel</code>	logado	—	cartões, últimas movimentações, críticos
<code>listarMateriais</code>	logado	q, categoria, status	lista com estoque total, saldo por depósito e status
<code>categorias</code>	logado	—	lista de categorias
<code>criarMaterial</code>	logado	nome, categoria, unidade, minimo	material criado (código automático)
<code>registrarEntrada</code>	logado	material, quantidade, local, nota, data, obs	movimentação
<code>registrarSaida</code>	logado	material, quantidade, obra, data, responsavel, obs, local	movimentação
<code>registrarTransferencia</code>	logado	material, quantidade, origem, destino, obs	movimentação
<code>listarMovimentacoes</code>	logado	tipo, inicio, fim, busca, page, limit	itens, total, páginas e resumo por tipo
<code>ultimasMovimentacoes</code>	logado	tipo (<i>entradas, saidas, transferencias</i>)	últimas cinco
<code>distribuicao</code>	logado	—	depósitos com seus materiais
<code>salvarLead</code>	público	dados do chatbot	lead salvo
<code>listarLeads</code>	logado	—	até 200 leads
<code>seed</code>	master key	—	carga inicial (também disponível como job)

6.1 Tratamento de erros

Erros de validação são lançados com o código Parse 141 e uma mensagem em JSON, interpretada pelo front-end:

```
{ "erro": "Estoque insuficiente",
  "campos": { "quantidade": "Estoque insuficiente (disponível: 18 sacos)" } }
```

As chaves de `campos` coincidem com os nomes dos campos dos formulários, de modo que a mensagem aparece junto ao campo correspondente.

7 Regras de negócio

7.1 Status do material

Com E sendo o estoque total (soma de todos os depósitos) e M o estoque mínimo:

$$\text{status} = \begin{cases} \text{ok}, & E \geq M \\ \text{crítico}, & E < 0,6 M \\ \text{baixo}, & \text{caso contrário.} \end{cases}$$

7.2 Entrada

Soma a quantidade ao saldo do material no depósito escolhido (padrão: Depósito A), criando o registro de estoque caso ele ainda não exista.

7.3 Saída

Exige obra de destino e quantidade positiva. O sistema retira primeiro dos depósitos com maior saldo, até completar a quantidade. Se o total disponível for menor que o solicitado, a operação é recusada e a mensagem informa o saldo disponível.

7.4 Transferência

Exige origem e destino válidos e diferentes, e saldo suficiente na origem. A quantidade é subtraída da origem e somada ao destino na mesma operação de gravação.

7.5 Histórico

A consulta é filtrada e paginada no servidor (8 itens por página por padrão, máximo de 100) e ordenada da mais recente para a mais antiga. A resposta inclui o total de registros por tipo (**resumo**), usado nos cartões da tela.

8 Autenticação e segurança

- O cadastro usa o e-mail como **username**. Senhas têm mínimo de 6 caracteres e são armazenadas com *hash* pelo Parse.
- Após o login, o servidor devolve um *token de sessão*, guardado no **localStorage** e enviado no cabeçalho **X-Parse-Session-Token**. Ao abrir o sistema, a função **me** restaura a sessão.
- Se o token expirar (código 209), o front-end o remove e recarrega a página, voltando à tela de login.

- As chaves **Application-Id** e **JavaScript-Key** são públicas por natureza e ficam no front-end. A **Master Key** nunca deve ser colocada no front-end; ela é usada apenas dentro do Cloud Code.
- As *Class Level Permissions* das classes de dados devem negar acesso público, para que as tabelas só sejam acessadas pelas Cloud Functions.
- A função **seed** exige a master key, e **salvarLead** é a única função de dados aberta ao público.

9 Integração do front-end

O arquivo `app/api.js` centraliza as chamadas. Ele monta a requisição, anexa o token de sessão quando existe e converte erros do Parse em exceções com o campo `campos`.

```
const BASE = process.env.NEXT_PUBLIC_PARSE_SERVER_URL
  || "https://parseapi.back4app.com";

export async function api(fn, params = {}) {
  const token = localStorage.getItem("token");
  const res = await fetch(`${BASE}/functions/${fn}`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-Parse-Application-Id": process.env.NEXT_PUBLIC_PARSE_APP_ID,
      "X-Parse-Javascript-Key": process.env.NEXT_PUBLIC_PARSE_JS_KEY,
      ...(token && { "X-Parse-Session-Token": token }),
    },
    body: JSON.stringify(params),
  });
  const data = await res.json();
  if (!res.ok) { /* converte data.error em Error com .campos */ }
  return data.result;
}
```

Exemplo de uso em uma tela (registro de entrada):

```
try {
  await api("registrarEntrada", { material, quantidade, local, nota, data, obs });
} catch (err) {
  setErrors(err.campos || { quantidade: err.message });
}
```

10 Instalação e implantação

10.1 Back-end (Back4App)

1. Criar um aplicativo no Back4App.
2. Em *Cloud Code* → *Functions*, enviar `backend/cloud/main.js` como `cloud/main.js` e clicar em *Deploy*.

3. Executar o job **seed** em *Server Settings* → *Core Settings* → *Background Jobs*. Ele cria 4 depósitos, 12 materiais e o estoque inicial, e não altera um banco que já tenha dados.
4. Em cada classe de dados, remover o acesso público nas *Class Level Permissions*.

10.2 Front-end

Criar o arquivo `.env.local` a partir de `.env.example`:

```
NEXT_PUBLIC_PARSE_APP_ID=seu_application_id
NEXT_PUBLIC_PARSE_JS_KEY=sua_javascript_key
NEXT_PUBLIC_PARSE_SERVER_URL=https://parseapi.back4app.com
```

Em seguida:

```
npm install
npm run dev      # desenvolvimento em http://localhost:3000
npm run build    # build de produção
```

Após alterar o `.env.local`, o servidor de desenvolvimento deve ser reiniciado.

10.3 Problemas comuns

Sintoma	Causa provável
Resposta 403 / <i>unauthorized</i>	APP_ID ou JS_KEY incorretos no <code>.env.local</code>
<i>Invalid function</i>	<code>main.js</code> não foi publicado com <i>Deploy</i>
Listas vazias	job seed não executado
Erro de rede	NEXT_PUBLIC_PARSE_SERVER_URL incorreta
Login falha após cadastro	verificação de e-mail ativada no app

11 Limitações e trabalhos futuros

- **Concorrência.** O Parse não permite descontar estoque condicionado ao saldo em uma única operação atômica. Duas saídas simultâneas do mesmo material podem ultrapassar o saldo. Para uso interno de baixo volume o risco é pequeno; uma alternativa é serializar as operações por material.
- **Perfis de acesso.** Todo usuário autenticado pode executar todas as operações. O campo **cargo** existe, mas ainda não restringe funções.
- **Recuperação de senha.** O botão “Esqueceu a senha?” ainda não tem função associada.
- **Chatbot.** O fluxo é uma simulação na própria interface e grava o *lead*. A integração real com a API do WhatsApp não está implementada.
- **Painel.** A data e a hora exibidas no cabeçalho do painel ainda são valores fixos no código.

- **Edição e exclusão.** Não há correção ou estorno de movimentações; o histórico é somente de inclusão.
- **Testes.** Não há testes automatizados; a validação foi feita manualmente.

12 Conclusão

O Marmopedras entrega o ciclo completo de controle de estoque — cadastro, consulta, movimentação e histórico — com regras de negócio centralizadas no servidor e um back-end enxuto, mantido em um único arquivo de Cloud Code. A arquitetura simples facilita a implantação no Back4App e deixa claros os próximos passos: perfis de acesso, operações atômicas de estoque e integração real com o WhatsApp.